

Framework authors know their own problems, and the problems of their coworkers and friends. And they write their frameworks to solve *those* problems—not yours.

Of course, your problems will likely overlap with those other problems quite a bit. If this were not the case, frameworks would not be so popular. To the extent that such overlap exists, frameworks can be very useful indeed.

ASYMMETRIC MARRIAGE

The relationship between you and the framework author is extraordinarily asymmetric. You must make a huge commitment to the framework, but the framework author makes no commitment to you whatsoever.

Think about this point carefully. When you use a framework, you read through the documentation that the author of that framework provides. In that documentation, the author, and other users of that framework, advise you on how to integrate your software with the framework. Typically, this means wrapping your architecture around that framework. The author recommends that you derive from the framework's base classes, and import the framework's facilities into your business objects. The author urges you to *couple* your application to the framework as tightly as possible.

For the framework author, coupling to his or her own framework is not a risk. The author *wants* to couple to that framework, because the author has absolute control over that framework.

What's more, the author wants *you* to couple to the framework, because once coupled in this way, it is very hard to break away. Nothing feels more validating to a framework author than a bunch of users willing to inextricably derive from the author's base classes.

In effect, the author is asking you to marry the framework—to make a huge, long-term commitment to that framework. And yet, under no circumstances will the author make a corresponding commitment to you. It's a one-directional marriage. You take on all the risk and burden; the framework author takes on nothing at all.

THE RISKS

What are the risks? Here are just a few for you to consider.

- The architecture of the framework is often not very clean. Frameworks tend to violate the Dependency Rule. They ask you to inherit their code into your business objects—your Entities! They want their framework coupled into that innermost circle. Once in, that framework isn't coming back out. The wedding ring is on your finger; and it's going to stay there.
- The framework may help you with some early features of your application. However, as your product matures, it may outgrow the facilities of the framework. If you've put on that wedding ring, you'll find the framework fighting you more and more as time passes.
- The framework may evolve in a direction that you don't find helpful. You may be stuck upgrading to new versions that don't help you. You may even find old features, which you made use of, disappearing or changing in ways that are difficult for you to keep up with.
- A new and better framework may come along that you wish you could switch to.

THE SOLUTION

What is the solution?

Don't marry the framework!

Oh, you can *use* the framework—just don't couple to it. Keep it at arm's length. Treat the framework as a detail that belongs in one of the outer circles of the architecture. Don't let it into the inner circles.

If the framework wants you to derive your business objects from its base classes, say no! Derive proxies instead, and keep those proxies in components that are *plugins* to your business rules.

Don't let frameworks into your core code. Instead, integrate them into components that plug in to your core code, following the Dependency Rule.

For example, maybe you like Spring. Spring is a good dependency injection framework. Maybe you use Spring to auto-wire your dependencies. That's fine, but you should not sprinkle `@autowired` annotations all throughout your business objects. Your business objects should not know about Spring.

Instead, you can use Spring to inject dependencies into your `Main` component. It's OK for `Main` to know about Spring since `Main` is the dirtiest, lowest-level component in the architecture.

I NOW PRONOUNCE YOU ...

There are some frameworks that you simply must marry. If you are using C++, for example, you will likely have to marry STL—it's hard to avoid. If you are using Java, you will almost certainly have to marry the standard library.

That's normal—but it should still be a *decision*. You must understand that when you marry a framework to your application, you will be stuck with that framework for the rest of the life cycle of that application. For better or for worse, in sickness and in health, for richer, for poorer, forsaking all others, you *will* be using that framework. This is not a commitment to be entered into lightly.

CONCLUSION

When faced with a framework, try not to marry it right away. See if there aren't ways to date it for a while before you take the plunge. Keep the framework behind an architectural boundary if at all possible, for as long as possible. Perhaps you can find a way to get the milk without buying the cow.

CASE STUDY: VIDEO SALES



Now it's time to put these rules and thoughts about architecture together into a case study. This case study will be short and simple, yet will depict both the process a good architect uses and the decisions that such an architect makes.

THE PRODUCT

For this case study, I've chosen a product with which I am rather intimately familiar: the software for a website that sells videos. Of course, it is reminiscent of cleancoders.com, the site where I sell my software tutorial videos.

The basic idea is trivial. We have a batch of videos we want to sell. We sell them, on the web, to both individuals and businesses. Individuals can pay one price to stream